

# AutoCam Tracker V1

## 開發日誌 - 前端組 / 後端組分工

版本範圍：MacBook 電腦鏡頭 + YOLOv11 + 數位變焦追蹤

文件用途：明確定義 V1 階段前端組與後端組的責任、交接資料格式、週期任務與整合完成標準。

### 1. V1 開發目標

V1 階段目標是先完成一套可展示、可操作、可驗證核心追蹤流程的 macOS C++ App。此版本不接 Sony 相機、不接 DJI 穩定器、不接 CAN，改用 MacBook Pro 內建鏡頭作為 Live View，並用數位變焦 / Digital Crop 模擬未來穩定器追蹤效果。

- 核心流程：電腦鏡頭取像 -> YOLOv11 偵測 -> 顯示車輛縮圖 -> 使用者點選目標 -> 單一目標追蹤 -> 計算中心偏移 -> 數位變焦置中。
- 開發分工：前端組負責使用者介面與互動，後端組負責影像、AI、追蹤、裁切與核心資料流程。
- V1 完成後，V2 可將 Digital Crop 的控制邏輯替換或延伸到實體 DJI 穩定器 / CAN 控制。

### 2. 組別責任總覽

| 組別  | 主要責任                                                                                                                               | 一句話定義                                |
|-----|------------------------------------------------------------------------------------------------------------------------------------|--------------------------------------|
| 前端組 | macOS App UI、Live View 顯示、偵測框繪製、車輛縮圖清單、按鈕與狀態顯示、使用者操作事件                                                                             | 把系統做成「人可以操作、看得懂、點得到目標」的 App。         |
| 後端組 | MacBook 鏡頭輸入、YOLOv11 推論、Detection 資料、Thumbnail 裁切、Target Selector、SimpleTracker、FramingController、DigitalCropController、Config、Log | 把系統做成「看得到畫面、偵測得到目標、追蹤得住、裁切能置中」的核心引擎。 |

### 3. 前端組工作內容

#### 3.1 建立 macOS App 介面

- 建立 Qt 6 / C++ App 介面與 MainWindow。
- 建立 Live View 顯示區、右側車輛縮圖清單、下方狀態列與控制面板。
- 畫面布局建議：左側為主 Live View，右側為 Detected Vehicles，底部顯示 FPS、Tracking Status、error\_x、error\_y。

#### 3.2 Live View 顯示

- 顯示 MacBook 鏡頭原始畫面 Raw View。

- 繪製 YOLO bbox、目前被鎖定的目標框、中心框與 dead zone。
- 顯示 Digital Crop 後的追蹤畫面 Cropped / Tracking View。
- V1 可先做單一主畫面，後續再擴充為原始畫面與追蹤畫面雙視窗。

### 3.3 車輛縮圖清單

- 右側建立 Detected Vehicles Panel。
- 每個項目顯示車輛縮圖、Detection ID、Confidence、是否被選中、Active / Lost 狀態。
- 使用者可點擊縮圖設定追蹤目標、取消追蹤或重新選擇另一台車。

### 3.4 操作按鈕與控制面板

- 基本按鈕：Start Camera、Stop Camera、Start Detection、Stop Detection、Lock Target、Unlock Target、Enable Digital Zoom、Disable Digital Zoom、Reset Tracking。
- 參數控制：Confidence Threshold、Dead Zone Size、Digital Zoom Ratio、Tracking Smoothing、Lost Timeout。
- 前端設定變更需透過統一事件或 API 傳給後端，不直接修改後端內部狀態。

### 3.5 狀態顯示與 Debug UI

- 顯示 FPS、YOLO inference time、目前追蹤狀態、目前目標 ID。
- 顯示 error\_x、error\_y、normalized\_error\_x、normalized\_error\_y。
- 顯示 crop window x/y、confidence、target lost 狀態。
- 追蹤狀態建議：Idle、Detecting、Target Selected、Tracking、Target Lost、Error。

## 4. 後端組工作內容

### 4.1 建立核心 Pipeline

- MacBook 鏡頭取得畫面。
- YOLOv11 偵測並產生 bbox。
- 裁切車輛 thumbnail 給前端顯示。
- 接收前端選擇的目標 ID。
- SimpleTracker 維持追蹤。
- FramingController 計算中心偏移。
- DigitalCropController 做數位變焦置中。
- 輸出畫面與狀態給前端。

### 4.2 影像輸入模組

- 建立 IVideoSource interface。
- 建立 MacCameraSource 讀取 MacBook 內建鏡頭。
- 建立 VideoFileSource，方便影片檔測試 YOLO 與 Tracking。
- 支援解析度與 FPS 設定，並處理鏡頭開啟失敗。

### 4.3 YOLOv11 推論模組

- 準備 YOLOv11 ONNX 模型。
- 建立 YoloDetector，使用 ONNX Runtime C++ 載入模型。
- 完成 image preprocessing、inference、postprocessing。
- 輸出 bbox、label、confidence。
- V1 可先偵測 car、motorcycle、person，之後再換自訓模型。

### 4.4 Thumbnail 裁切模組

- 根據 bbox 從 frame 裁切 thumbnail。
- 避免 bbox 超出畫面邊界。
- 統一 thumbnail 尺寸，並綁定 detection id。
- 處理裁切失敗或 bbox 無效情況。

### 4.5 Target Selector 與 SimpleTracker

- TargetSelector 接收 selectTarget(detectionId)，設定 selectedTargetId，支援 unlockTarget() 與 resetTracking()。
- SimpleTracker 使用上一幀 target bbox，在下一幀 detections 中尋找中心距離最接近的 bbox。
- 若距離小於門檻，視為同一台車；若找不到，進入 lost 狀態。
- lost 超過指定時間後顯示 target lost。

### 4.6 FramingController

- 計算 frame center、target center、error\_x、error\_y。
- 計算 normalized error，建立 dead zone。
- 輸出虛擬 pan / tilt command，供 DigitalCropController 使用，也為 V2 的實體雲台控制預留資料格式。

### 4.7 DigitalCropController

- 根據 error\_x / error\_y 移動 crop window。
- 加入 smoothing，避免畫面跳動。
- 限制 crop window 不超出原始 frame 邊界。
- 輸出裁切後畫面，讓目標盡量維持在畫面中心。

### 4.8 Config 與 Log

- 建立 default\_config.json 與 ConfigManager。
- 設定項目包含 camera\_index、frame\_width、frame\_height、fps、yolo\_model\_path、confidence\_threshold、iou\_threshold、lost\_timeout\_ms、max\_center\_distance、dead\_zone、digital\_zoom\_ratio、smoothing。
- 使用 spdlog 或同等工具記錄鏡頭錯誤、YOLO 載入錯誤、inference 錯誤、tracking lost、digital crop 邊界錯誤、FPS 與 inference time。

## 5. 前後端交接介面

### 5.1 後端提供給前端

- raw frame。
- display frame / cropped frame。
- detections 與 thumbnails。
- selected target id。
- tracking status。
- FPS 與 inference time。
- error\_x、error\_y。
- crop window position。

### 5.2 前端傳給後端

- 開始鏡頭、停止鏡頭。
- 開始偵測、停止偵測。
- 選擇目標 detection id。
- 取消追蹤、重置追蹤。
- 開關數位變焦。
- 修改 confidence threshold、dead zone、zoom ratio、smoothing。

## 6. 建議資料結構

前後端必須先約定資料格式，才能並行開發。以下是 V1 建議的介面資料結構。

```
struct UiDetectionItem {
    int id;
    std::string label;
    float confidence;
    cv::Rect bbox;
    cv::Mat thumbnail;
    bool selected;
    bool active;
};

struct UiFrameData {
    cv::Mat rawFrame;
    cv::Mat displayFrame;
    std::vector<UiDetectionItem> detections;
    int selectedTargetId;
    float fps;
    float inferenceTimeMs;
    float errorX;
    float errorY;
    std::string trackingStatus;
};
```

## 7. 三週開發任務規劃

| 週次                   | 前端組任務                                                                                                                                                                              | 後端組任務                                                                                                                                                                                                              |
|----------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 第一週 - 建立骨架與假資料流程     | <ul style="list-style-type: none"><li>• 建立 MainWindow、LiveViewWidget、VehicleListWidget、StatusBarWidget。</li><li>• 前端用假資料顯示 bbox 與車輛縮圖，完成點擊縮圖事件與 Start / Stop / Reset 按鈕。</li></ul> | <ul style="list-style-type: none"><li>• 建立 C++ 專案核心資料結構、IVideoSource、MacCameraSource、VideoFileSource。</li><li>• 建立 Detection struct、假 YoloDetector interface、FramingController、DigitalCropController 雛形。</li></ul> |
| 第二週 - 串接真實影像與 YOLO   | <ul style="list-style-type: none"><li>• 串接後端 frame，顯示真實 MacBook 鏡頭畫面。</li><li>• 顯示後端 detections、thumbnail list、tracking status、error_x、error_y。</li></ul>                          | <ul style="list-style-type: none"><li>• 接 YOLOv11 ONNX Runtime，完成 preprocessing、postprocessing。</li><li>• 輸出真實 detections，完成 thumbnail 裁切、TargetSelector、SimpleTracker 第一版。</li></ul>                              |
| 第三週 - 完成數位追蹤與整合 Demo | <ul style="list-style-type: none"><li>• 完成 digital crop 畫面顯示。</li><li>• 加入原始畫面 / 追蹤畫面切換，加入 confidence threshold、dead zone、zoom ratio UI，優化狀態顯示。</li></ul>                          | <ul style="list-style-type: none"><li>• 完成 DigitalCropController、smoothing、lost target timeout。</li><li>• 加入 ConfigManager、Logger，修正 tracking 穩定性，整合 V1 demo pipeline。</li></ul>                                   |

## 8. V1 前後端整合完成標準

1. App 可以開啟 MacBook 鏡頭。
2. Live View 可以顯示即時畫面。
3. YOLO 可以偵測畫面中的車輛 / 物件。
4. 右側可以顯示偵測到的車輛縮圖。
5. 使用者可以點擊其中一張縮圖。
6. 系統可以鎖定單一追蹤目標。
7. 畫面上可以顯示中心框與目標框。
8. 系統可以計算 error\_x / error\_y。
9. Digital Crop 可以讓目標維持接近中心。
10. UI 可以顯示 FPS、tracking status、inference time。
11. 系統可以處理 target lost。
12. 所有功能都透過 GitHub PR 合併。

## 9. 分工結論

- 前端組：負責把 V1 系統做成可操作、可展示、可調整參數的 macOS App。
- 後端組：負責把 V1 系統做成可取像、可偵測、可追蹤、可計算偏移、可數位裁切置中的核心引擎。
- 兩組開發時必須先固定資料格式。前端可以先用假資料開發 UI，後端可以先用影片檔和假 UI 測試 pipeline，最後再整合。